Guru Ghasidas Vishwavidyalaya

Session: 2026

Major:

Object Oriented Programming with Java

Assignment for the fulfillment of Lab Course

SUBMITTED TO	SUBMITTED BY
Dr. Ratnesh Shrivastava	Om Nagose
Associate Professor	BSc. (H) Computer Science

4th Semester GGV/24/05145

Dept. of Computer Science & Information Technology

1. *Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.*

```
import java.io.*;
import java.util.Scanner;

class Farmer {
  String name, fruit, city;
  int quantity;
}

public class Main {
  public static void main(String [] args) throws Exception {
    Scanner sc = new Scanner(System.in);

    Farmer[] f = new Farmer[10];

    // Input
    for (int i = 0; i < 10; i++) {
      f[i] = new Farmer();
      System.out.println("Farmer " + (i + 1));

      System.out.print("Name: ");
      f[i].name = sc.nextLine();

      System.out.print("Fruit: ");
      f[i].fruit = sc.nextLine();

      System.out.print("City: ");
      f[i].city = sc.nextLine();

      System.out.print("Quantity: ");
      f[i].quantity = sc.nextInt();
      sc.nextLine(); // clear buffer

      System.out.println();
    }
```

```
// Save BEFORE sorting
FileWriter fw = new FileWriter("farmers.txt");
fw.write("Before Sorting:\n\n");
fw.write("Name\tFruit\tCity\tQuantity\n");

for (int i = 0; i < 10; i++) {
   fw.write(f[i].name + "\t" + f[i].fruit + "\t" + f[i].city + "\t" + f[i].quantity + "\n");
}

// -------- Bubble Sort (simple sorting) --------
for (int i = 0; i < 9; i++) {
   for (int j = 0; j < 9 - i; j++) {
      if (f[j].quantity > f[j + 1].quantity) {
         Farmer temp = f[j];
         f[j] = f[j + 1];
         f[j + 1] = temp;
      }
   }
}

// Save AFTER sorting
fw.write("\nAfter Sorting (by Quantity):\n\n");
fw.write("Name\tFruit\tCity\tQuantity\n");

for (int i = 0; i < 10; i++) {
   fw.write(f[i].name + "\t" + f[i].fruit + "\t" + f[i].city + "\t" + f[i].quantity + "\n");
}

fw.close();

// Display sorted data
System.out.println("\nSorted Data:");
System.out.println("Name\tFruit\tCity\tQuantity");

for (int i = 0; i < 10; i++) {
   System.out.println(f[i].name + "\t" + f[i].fruit + "\t" + f[i].city + "\t" + f[i].quantity);
}

// Search
System.out.print("\nEnter farmer name to search: ");
String search = sc.nextLine();
```

```
        boolean found = false;
        for (int i = 0; i < 10; i++) {
            if (f[i].name.equalsIgnoreCase(search)) {
                System.out.println("Farmer Found:");
                System.out.println(f[i].name + "\t" + f[i].fruit + "\t" + f[i].city + "\t" + f[i].quantity);
                found = true;
                break;
            }
        }

        if (!found) {
            System.out.println("Farmer not found!");
        }

        sc.close();
    }
}
```

Input:

Farmer 1

Name: Rahul Mourya

Fruit: Mango

City: Delhi

Quantity: 50

Farmer 2

Name: Aman

Fruit: Apple

City: Mumbai

Quantity: 30

Farmer 3

Name: Sita

Fruit: Banana

City: Kolkata

Quantity: 40

Farmer 4

Name: Mohan

Fruit: Orange

City: Jaipur

Quantity: 25

Farmer 5

Name: Neha

Fruit: Grapes

City: Pune

Quantity: 60

Farmer 6

Name: Raj

Fruit: Pineapple

City: Goa

Quantity: 20

Farmer 7

Name: Kiran

Fruit: Papaya

City: Bhopal

Quantity: 25

Farmer 8

Name: Arjun

Fruit: Guava

City: Lucknow

Quantity: 45

Farmer 9

Name: Pooja

Fruit: Watermelon

City: Indore

Quantity: 55

Farmer 10

Name: Vikram

Fruit: Cherry

City: Shimla

Quantity: 15

Sorted Data:

Name Fruit City Quantity

Vikram Cherry Shimla 15

Raj Pineapple Goa 20

Mohan Orange Jaipur 25

Kiran Papaya Bhopal 25

Aman Apple Mumbai 30

Sita Banana Kolkata 40

Arjun Guava Lucknow 45

Rahul Mourya Mango Delhi 50

Pooja Watermelon Indore 55

Neha Grapes Pune 60

Output:

Enter farmer name to search: Arjun

Farmer Found:

Arjun Guava Lucknow 45

Search input and output:

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

```
import java.util.Scanner;


public class FruitData {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);


    String[] fruits = {"Mango", "Apple", "Papaya"};
    int[] quantity = new int[3];
    String[] quality = new String[3];


    // Input
    for (int i = 0; i < 3; i++) {
      System.out.println("Enter quantity of " + fruits[i] + ": ");
      quantity[i] = sc.nextInt();


      // If quantity > 1000, store half
      if (quantity[i] > 1000) {
        quantity[i] = quantity[i] / 2;
      }


      System.out.println("Enter quality (good/average/bad) of " + fruits[i] + ": ");
      quality[i] = sc.next();
    }


    // Display data
    System.out.println("\n--- Stored Data ---");
```

```
for (int i = 0; i < 3; i++) {
   System.out.println(fruits[i] + " | Quantity: " + quantity[i] + " | Quality: " + quality[i]);
}

// Search
System.out.println("\nEnter fruit name to search: ");
String searchFruit = sc.next();

System.out.println("Enter quality to search: ");
String searchQuality = sc.next();

boolean found = false;

for (int i = 0; i < 3; i++) {
   if (fruits[i].equalsIgnoreCase(searchFruit) &&
      quality[i].equalsIgnoreCase(searchQuality)) {

      System.out.println("Found: " + fruits[i] +
                  " | Quantity: " + quantity[i] +
                  " | Quality: " + quality[i]);
      found = true;
   }
}

if (!found) {
   System.out.println("Fruit not found with given quality.");
}
```

```
        sc.close();
    }
}
```

Input

Enter quantity of Mango:
1200
Enter quality of Mango:
good

Enter quantity of Apple:
800
Enter quality of Apple:
average

Enter quantity of Papaya:
1500
Enter quality of Papaya:
bad

Output

Mango → 600 (halved)
Apple → 800
Papaya → 750 (halved)

Example Search

Search fruit: Mango
Search quality: good

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.

```
import java.util.Scanner;


public class OrchardSearch {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);


    String[] orchard = {"A", "B", "C", "D"};
    String[] fruit = {"Apple", "Litchi", "Mango", "Apple"};
    String[] quality = {"Good", "Good", "Good", "Bad"};


    // Display data
    System.out.println("Orchard Data:");
    for (int i = 0; i < 4; i++) {
      System.out.println("Orchard " + orchard[i] +
                    " | Fruit: " + fruit[i] +
                    " | Quality: " + quality[i]);
    }


    // Search
    System.out.println("\nEnter fruit to search:");
    String searchFruit = sc.next();


    System.out.println("Enter quality to search:");
    String searchQuality = sc.next();


    boolean found = false;
```

```
        for (int i = 0; i < 4; i++) {
            if (fruit[i].equalsIgnoreCase(searchFruit) &&
                quality[i].equalsIgnoreCase(searchQuality)) {

                System.out.println("Found in Orchard " + orchard[i]);
                found = true;
            }
        }

        if (!found) {
            System.out.println("No matching orchard found.");
        }

        sc.close();
    }
}
```

Example Search

Input:
Apple
Good

Output:

Found in Orchard A

4. 4. Take a map provided as a graph & display all the subtrees of connected points.

```
import java.util.Scanner;

public class GraphSubtrees {
  static int n;
  static int[][] graph;
  static boolean[] visited;

  // DFS function
  static void dfs(int node) {
    visited[node] = true;
    System.out.print(node + " ");

    for (int i = 0; i < n; i++) {
      if (graph[node][i] == 1 && !visited[i]) {
        dfs(i);
      }
    }
  }

  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter number of nodes:");
    n = sc.nextInt();

    graph = new int[n][n];
    visited = new boolean[n];
```

```
        System.out.println("Enter adjacency matrix:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = sc.nextInt();
            }
        }

        System.out.println("\nSubtrees (Connected Components):");

        for (int i = 0; i < n; i++) {
            if (!visited[i]) {
                System.out.print("Subtree: ");
                dfs(i);
                System.out.println();
            }
        }

        sc.close();
    }
}
```

Example Input

Enter number of nodes:
5

Adjacency Matrix:
0 1 0 0 0
1 0 0 0 0
0 0 0 1 1

0 0 1 0 1
0 0 1 1 0

Output

Subtree: 0 1
Subtree: 2 3 4

5. 5. Enter entry point & goal point & produce the matching subtree of the graph.

```
import java.util.Scanner;

public class GraphPath {
  static int n;
  static int[][] graph;
  static boolean[] visited;
  static int[] path;
  static int pathIndex = 0;
  static boolean found = false;

  static void dfs(int node, int goal) {
    visited[node] = true;
    path[pathIndex++] = node;

    if (node == goal) {
      found = true;
      return;
    }

    for (int i = 0; i < n; i++) {
      if (graph[node][i] == 1 && !visited[i]) {
        dfs(i, goal);
        if (found) return; // stop when goal found
      }
    }

    pathIndex--; // backtrack
```

```
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.println("Enter number of nodes:");
    n = sc.nextInt();

    graph = new int[n][n];
    visited = new boolean[n];
    path = new int[n];

    System.out.println("Enter adjacency matrix:");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            graph[i][j] = sc.nextInt();
        }
    }

    System.out.println("Enter entry node:");
    int start = sc.nextInt();

    System.out.println("Enter goal node:");
    int goal = sc.nextInt();

    dfs(start, goal);

    if (found) {
```

```
        System.out.print("Matching Subtree (Path): ");
        for (int i = 0; i < pathIndex; i++) {
            System.out.print(path[i] + " ");
        }
    } else {
        System.out.println("No path found.");
    }

    sc.close();
  }
}
```

Example Input

Nodes: 5

Adjacency Matrix:
0 1 0 0 0
1 0 1 0 0
0 1 0 1 1
0 0 1 0 0
0 0 1 0 0

Entry: 0
Goal: 4

Output

Matching Subtree (Path): 0 1 2 4

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

```
import java.util.Scanner;


public class ShortestPath {
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);


    int n;
    System.out.println("Enter number of nodes:");
    n = sc.nextInt();


    int[][] graph = new int[n][n];


    System.out.println("Enter weighted adjacency matrix (0 if no edge):");
    for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
        graph[i][j] = sc.nextInt();
      }
    }


    System.out.println("Enter start node:");
    int start = sc.nextInt();


    System.out.println("Enter end node:");
    int end = sc.nextInt();


    int[] dist = new int[n];
```

```
boolean[] visited = new boolean[n];
int[] parent = new int[n];

// Initialize
for (int i = 0; i < n; i++) {
   dist[i] = Integer.MAX_VALUE;
   parent[i] = -1;
}

dist[start] = 0;

// Dijkstra
for (int count = 0; count < n - 1; count++) {
   int min = Integer.MAX_VALUE, u = -1;

   // Find minimum distance node
   for (int i = 0; i < n; i++) {
      if (!visited[i] && dist[i] < min) {
         min = dist[i];
         u = i;
      }
   }

   visited[u] = true;

   // Update neighbors
   for (int v = 0; v < n; v++) {
      if (graph[u][v] != 0 && !visited[v] &&
```

```
                dist[u] + graph[u][v] < dist[v]) {

                dist[v] = dist[u] + graph[u][v];
                parent[v] = u;
            }
        }
    }

    // Print shortest distance
    System.out.println("Shortest Distance: " + dist[end]);

    // Print path
    System.out.print("Shortest Path: ");
    printPath(parent, end);

    sc.close();
  }

  static void printPath(int[] parent, int j) {
    if (j == -1) return;
    printPath(parent, parent[j]);
    System.out.print(j + " ");
  }
}
```

Example Input

Nodes: 5

Weighted Matrix:
0 2 0 0 0

2 0 3 0 0
0 3 0 4 6
0 0 4 0 1
0 0 6 1 0

Start: 0
End: 4

Output

Shortest Distance: 10
Shortest Path: 0 1 2 3 4

7. Write a program to demonstrate the use of all keywords made for exception handling.

```
import java.util.Scanner;


class ExceptionDemo {


  // using 'throws'
  static void checkNumber(int num) throws Exception {
    if (num < 0) {
      // using 'throw'
      throw new Exception("Number cannot be negative");
    } else {
      System.out.println("Valid number: " + num);
    }
  }


  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);


    try { // try block
      System.out.println("Enter a number:");
      int num = sc.nextInt();


      checkNumber(num); // may throw exception


      int a = 10, b;
      System.out.println("Enter divisor:");
      b = sc.nextInt();
```

```
        int result = a / b; // may cause ArithmeticException
        System.out.println("Result: " + result);

    }
    catch (ArithmeticException e) { // specific catch
        System.out.println("Error: Cannot divide by zero");
    }
    catch (Exception e) { // general catch
        System.out.println("Exception: " + e.getMessage());
    }
    finally { // always executes
        System.out.println("Finally block executed");
    }

    sc.close();
  }
}
```

Example Runs

Case 1 (Negative number)

Input: -5
Output: Exception: Number cannot be negative
Finally block executed

Case 2 (Divide by zero)

Input: 10, 0
Output: Error: Cannot divide by zero
Finally block executed

Case 3 (Normal)

Input: 10, 2
Output: Result: 5
Finally block executed

8. 8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

```
import java.util.Scanner;


// Custom Exception
class NoMoneyException extends Exception {
  public NoMoneyException(String msg) {
    super(msg);
  }
}


public class BankAccount {


  static void withdraw(int balance, int amount) throws NoMoneyException {
    if (amount > balance) {
      throw new NoMoneyException("Insufficient balance!");
    } else {
      balance = balance - amount;
      System.out.println("Withdrawal successful.");
      System.out.println("Remaining balance: " + balance);
    }
  }


  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);


    try {
      System.out.println("Enter balance:");
```

```
        int balance = sc.nextInt();

        System.out.println("Enter withdrawal amount:");
        int amount = sc.nextInt();

        withdraw(balance, amount);

    } catch (NoMoneyException e) {
        System.out.println("Exception: " + e.getMessage());
    } finally {
        System.out.println("Transaction ended.");
    }

    sc.close();
  }
}
```

Example Runs

Case 1 (Not enough money)

Balance: 500
Withdraw: 800

Output:

Exception: Insufficient balance!
Transaction ended.

Case 2 (Valid)

Balance: 1000
Withdraw: 300

Output:

Withdrawal successful.
Remaining balance: 700
Transaction ended.

9. Write a program which enables to use a platform by a train one at a time.

```
class Platform {

  // synchronized method → only one thread allowed
  synchronized void usePlatform(String trainName) {
    System.out.println(trainName + " is entering platform");

    try {
      Thread.sleep(2000); // simulate time on platform
    } catch (InterruptedException e) {
      System.out.println(e);
    }

    System.out.println(trainName + " is leaving platform\n");
  }
}

// Train thread
class Train extends Thread {
  Platform p;
  String name;

  Train(Platform p, String name) {
    this.p = p;
    this.name = name;
  }

  public void run() {
```

```
        p.usePlatform(name);
    }
}

public class RailwaySystem {
    public static void main(String[] args) {

        Platform p = new Platform();

        Train t1 = new Train(p, "Train A");
        Train t2 = new Train(p, "Train B");
        Train t3 = new Train(p, "Train C");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Output (Order may vary)

```
Train A is entering platform
Train A is leaving platform

Train B is entering platform
Train B is leaving platform

Train C is entering platform
Train C is leaving platform
```

10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

```
class Restaurant {
  private boolean orderPlaced = false;
  private String order;

  // Waiter places order
  synchronized void placeOrder(String item) {
    try {
      while (orderPlaced) {
        wait(); // wait if kitchen busy
      }

      order = item;
      System.out.println("Waiter placed order: " + item);

      orderPlaced = true;
      notify(); // notify chef

    } catch (InterruptedException e) {
      System.out.println(e);
    }
  }

  // Chef prepares order
  synchronized void prepareOrder() {
    try {
      while (!orderPlaced) {
```

```
            wait(); // wait if no order
        }

        System.out.println("Chef is preparing: " + order);

        Thread.sleep(2000); // cooking time

        System.out.println("Chef delivered: " + order + "\n");

        orderPlaced = false;
        notify(); // notify waiter

      } catch (Exception e) {
        System.out.println(e);
      }
    }
}

// Waiter Thread
class Waiter extends Thread {
    Restaurant r;

    Waiter(Restaurant r) {
      this.r = r;
    }

    public void run() {
      String[] orders = {"Pizza", "Burger", "Pasta"};
```

```
        for (String item : orders) {
            r.placeOrder(item);
        }
    }
}

// Chef Thread
class Chef extends Thread {
    Restaurant r;

    Chef(Restaurant r) {
        this.r = r;
    }

    public void run() {
        for (int i = 0; i < 3; i++) {
            r.prepareOrder();
        }
    }
}

public class RestaurantSystem {
    public static void main(String[] args) {
        Restaurant r = new Restaurant();

        Waiter w = new Waiter(r);
        Chef c = new Chef(r);
```

```
        w.start();
        c.start();
    }
}
```

Sample Output

```
Waiter placed order: Pizza
Chef is preparing: Pizza
Chef delivered: Pizza

Waiter placed order: Burger
Chef is preparing: Burger
Chef delivered: Burger

Waiter placed order: Pasta
Chef is preparing: Pasta
Chef delivered: Pasta
```